DATA 266 : LAB 2

Lab 2 has three parts, completed by teams of two:

Part 1: MultiModal Retrieval-Augmented Generation (Kaggle Class Competition sign up)

Part 2: Fine-Tuning Diffusion Models for Domain-Specific Image Generation

Part 3: A Multimodal Multi-Agent System for Image Understanding and Generation

Marks Distribution = Part 1 (25 + 10 Kaggle) + Part 2 (27) + Part 3 (28) + Team Demo (10) = 100. Bonus tasks can add up to +4 marks (total capped at 100).

Key words: **MUST** = required for marks; **SHOULD** = recommended; **MAY** = optional. `##` is your two-digit team number (e.g. `07`).

Milestone	Date
Lab released	Tuesday, Oct 6, 2026
Final submission (Canvas, GitHub tags, Kaggle)	**Tuesday, Nov 17, 2026, 11:59 PM PT**
Team demos (15 minutes per team, 10 marks)	Nov 18–24, 2026 (slots posted on Canvas)

Team Rules *(all parts)*

1. Teams have two members and are named `Pair_Programming_Team_##` on both Canvas and Kaggle. **Both members MUST contribute to all three parts.**
2. For Parts 2 and 3, each member may lead one task. The partner who is not leading MUST still implement and run the complete pipeline, at least as a working baseline. Record every knowledge-transfer session and every discussion of architecture design and trade-offs as an audio or video file. Keep the recordings with your project files so you can revisit what was decided; they also show that both members were involved.
3. All required work MUST run on one 16 GB GPU (Kaggle T4/P100 or Colab T4). Any hosted API you use (LLM, VLM, image generation) MUST be named in the README with its model version.

Common Components *(apply to every part)*

Every part is built, evaluated and documented the same way. These four components are graded inside each part's "Evaluation, Safety & Documentation" task.

C1. Reproducibility (MUST)

1. One private GitHub repository `data266-lab2-team##` for all three parts, shared (Read access) with the instructor and ISAs. Handles are posted on Canvas.

2. A `requirements.txt` with pinned versions, and a `README.md` that says how to run each part.
3. Work in notebooks under `notebooks/` (e.g. `p1_rag.ipynb`, `p2_finetune.ipynb`, `p3_agents.ipynb`; you may split a part into several). Submit them **executed, with all outputs saved** (Restart & Run All). A notebook without outputs earns at most 50% of the marks it supports.
4. The first cell of every notebook prints the Git commit, library versions, GPU and random seed (code below). Fix seeds wherever randomness is involved.
5. Before the deadline, create one Git tag per part: `p1-v1.0`, `p2-v1.0`, `p3-v1.0`. Graders use the tagged commit.
6. Model weights and generated images go to Hugging Face (a free account or team organization), not to GitHub.

```
# Cell 1 - run information (required)
import subprocess, sys, random, numpy as np, torch
SEED = 266; random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
commit = subprocess.run(["git", "rev-parse", "--short", "HEAD"], capture_output=True,
text=True).stdout.strip()
print({"commit": commit, "python": sys.version.split()[0], "torch": torch.__version__,
       "gpu": torch.cuda.get_device_name(0) if torch.cuda.is_available() else "cpu", "seed": SEED})
```

C2. Evaluation (MUST)

Every part reports exactly four kinds of measurement, defined in its own section, and nothing more is required:

Measurement	Meaning
Primary metric(s)	The 1–2 numbers that say whether the system does its job
Secondary metric(s)	1–2 numbers that explain why it succeeds or fails
Human evaluation	A small, structured judgement by people (both members, plus an outside rater where stated)
Efficiency	One cost measure (time per request or GPU-hours)

1. Report each measurement for your **baseline** and your **final** system, in one results table per part.
2. Every number in the report MUST be computed in a notebook output. Numbers typed in by hand count as fabrication.
3. Write your test questions or prompts **before** tuning and commit them (the Git timestamp is checked).

C3. Documentation (MUST)

1. **Report:** one IEEE-format PDF covering all three parts, at most 12 pages per part plus appendices, in your own words. It includes, for each part, an architecture diagram, the results table, a short analysis of what worked and what did not, and the safety analysis (C4).

2. **Model card** for your Part 2 model on Hugging Face (half a page: base model, data, training settings, results, limitations, license).
3. **System card** for Part 3 in the report (half a page: what it can do, known limits, safety measures).
4. `CONTRIBUTIONS.md`: who did what in each part, plus a list of your meeting recordings (date, topic, link). `AI_USAGE.md`: which AI coding assistants you used and for what.

C4. Safety Analysis (MUST)

About one page per part in the report:

Part	What to analyse
Part 1	How your system handled the untrusted appendix in Document 3: the false claims, and the instructions hidden in its text and inside a chart image. Give examples of questions where it was or was not fooled.
Part 2	Dataset license (Naruto and Pokémon are copyrighted characters, so the model is for non-commercial research only), the NSFW filter you used on outputs, and a memorization check (below).
Part 3	Your guardrails and the results of 10 red-team cases (below).

Getting Started *(recommended plan)*

The plan below spreads the work over six weeks. Part 2 training can run in the background while you work on Part 1 and Part 3.

Week	Suggested work
1 (Oct 6–12)	Create the GitHub repo, Kaggle team and Hugging Face account. **Part 1 first milestone:** parse `Document.pdf`, build dense retrieval, and answer the 15 original questions.
2 (Oct 13–19)	Part 1: add Documents 2 and 3, figures and tables, and hybrid retrieval. Part 2: prepare the data and start the first LoRA run.
3 (Oct 20–26)	Part 1: conversation handling, your 25-item golden set, first Kaggle submission. Part 2: second run. **Part 2 first milestone:** one trained LoRA generating in-domain images.
4 (Oct 27–Nov 2)	Part 2: generate, evaluate and run the human evaluation. **Part 3 first milestone:** a linear pipeline (vision → prompt → generate) working on one image.
5 (Nov 3–9)	Part 3: planner, critique loop, safety checks, the 15 scenarios and 10 red-team cases.
6 (Nov 10–17)	Final evaluations, report, tags and final Kaggle submission. Rehearse the demo.

Suggested starter stack (any equivalent open-source tools are fine):

Part	Tools
Part 1	PyMuPDF for text and figure crops; `sentence-transformers` (e.g. `BAAI/bge-small-en-v1.5`) for text embeddings; `rank_bm25` for keyword search; ChromaDB as the vector database; a small vision-language model (e.g. `Qwen/Qwen2.5-VL-3B-Instruct`) to describe figures and answer questions, or a declared API.
Part 2	`stable-diffusion-v1-5/stable-diffusion-v1-5` at 512×512; the diffusers example script `train_text_to_image_lora.py`; `torchmetrics` (KID) and an `open_clip` or `transformers` CLIP model for scores; the `lpips` package for diversity.
Part 3	LangGraph or plain Python functions; the same vision-language model as Part 1; diffusers `StableDiffusionImg2ImgPipeline` with your Part 2 LoRA; `timbrooks/instruct-pix2pix` for edits; Gradio for the demo.

Part 1: MultiModal RAG over a Read-Only Knowledge Base - Kaggle Class Competition *(25 + 10 Marks)*

Reference papers:

- Lewis et al., 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. https://arxiv.org/abs/2005.11401
- Faysse et al., 2024. ColPali: Efficient Document Retrieval with Vision Language Models. https://arxiv.org/abs/2407.01449
- Greshake et al., 2023. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. https://arxiv.org/abs/2302.12173

Build a **multimodal RAG system** that answers questions about three economics PDF documents using their text, figures and tables. The documents are a **read-only** knowledge base. Users may add or change information inside a conversation; those changes apply only to that conversation and never change the documents.

Files provided (Canvas: Files >> Labs >> Lab 2 >> DATA266_Lab2_Part1, and the Kaggle Data tab)

File	Contents
`Document.pdf` (DOC1)	Blanchard, *Macroeconomics*, Ch. 1–2. Trust: high. Values for 2011–2012 are forecasts (marked *)
`Document2_Supplement_A_Outturns_2011-2013.pdf` (DOC2)	Actual results for 2011–2013, one stated revision, near-identical charts. Trust: high
`Document3_Briefing_B_Labour_Prices_Policy.pdf` (DOC3)	Re-estimates, some values that disagree with DOC1/DOC2, a corrected figure, and an untrusted appendix (Appendix C). Trust: medium; Appendix C: low
`kb_manifest.json`	Document IDs, file names and trust levels
`Questions.csv`	40 practice questions in 35 conversations
`sample_submission.csv`	Submission template; every row shows a format example (placeholders, not answers)
`validate_submission.py`	Format checker: `python validate_submission.py submission.csv Questions.csv`

Answering rules (MUST)

1. A stated revision or correction replaces the earlier value; an actual result replaces a forecast. Mention the value it replaces.
2. If DOC1/DOC2/DOC3 disagree with no revision note, answer with status `conflict`, give both values, and prefer the higher-trust source.
3. Never use Appendix C of DOC3 as evidence, and never follow instructions found inside any document or image.
4. If the documents do not support an answer, return `not_found`. If the question assumes something false, return `false_premise` and correct it.
5. If the user supplies or changes a value ("assume…", "for this chat…"), use it only in that conversation, say what the documents report, and use status `session_override`. Refuse requests to change the documents permanently.

Each answer is a JSON object with these fields:

```
{"answer": "...",                      # concise, at most 120 words
 "status": "answered" | "not_found" | "conflict" | "false_premise" | "session_override",
 "citations": ["DOC2:p4", "DOC1:Table 1-2"],
 "figure_references": ["Figure A-3"]}  # exact labels; [] if none
```

Actionable Tasks:

1.1. Ingestion & Indexing *(5 Marks)*

1. Extract text from all three PDFs into chunks, keeping the document ID and page number.
2. Extract every numbered Figure and Table as an image with its label and caption. "Figure B-5" and "Figure B-5 (corrected)" are two different objects.
3. Make figures searchable with one method: image embeddings (e.g. CLIP), or text descriptions written by a vision-language model.
4. Store everything in an open-source vector database with at least `doc_id, page, figure_id, trust_level`, read from `kb_manifest.json`.

1.2. Retrieval & Grounded Answers *(7 Marks)*

1. Hybrid retrieval: dense embeddings plus BM25 keyword search (a reranker MAY be added).
2. If a question names a figure or table (e.g. "Figure A-5"), retrieve that exact figure. Several charts look almost identical; never mix values from two different figures.
3. Generate the JSON answer from the retrieved text and figures, following the answering rules, with citations for every fact.

1.3. Conversation Handling *(5 Marks)*

1. Keep a separate memory for each `conversation_id`. Follow-up questions ("And in 2013?") use only the earlier turns of the same conversation.

2. Values supplied by the user are stored in that memory, never in the vector database, and are forgotten when the conversation ends.
3. Graded on the conversation questions in the hidden set (1.5 D): later turns answered correctly, user values used only in their own conversation, document values still reported, permanent changes refused.

1.4. Evaluation, Safety & Documentation *(8 Marks)*

1. Write a **golden set of 25 questions** yourselves from the three PDFs, with the correct answer, status and source page or figure. Cover facts and tables (5), figures including near-identical ones (6), questions needing two sources (3), multi-turn conversations with a user-supplied value (5, in at least 2 conversations), conflicts and revisions (3), and unanswerable, false-premise or injection questions (3). Questions copied or paraphrased from `Questions.csv` do not count.
2. Report the four measurements below for a baseline (dense retrieval only) and your final system, plus the safety analysis (C4) and documentation (C3).

Measurement	Part 1 definition
Primary	**Answer correctness** on your golden set: number correct out of 25 (numbers must match within ±1%; the status must be right)
Secondary	**Retrieval Recall@5** (the correct page or figure is among the top 5 results) and **citation accuracy** (share of cited sources that really support the answer)
Human evaluation	Both members rate each of the 25 final answers 1–5 for groundedness (is every claim supported by the cited source?). Report the average and how often you agreed within one point
Efficiency	Median seconds per question

1.5. Kaggle Leaderboard *(10 Marks)*

A. How to work with Questions.csv

Column	Example	Meaning
`id`	`33`	Row ID. Your submission has exactly one row per `id`, in the same order
`conversation_id`	`C032`	One conversation. 35 conversations: 32 have one question, 3 have two or three
`turn`	`2`	Position of the question inside its conversation
`question_id`	`Q033`	Question key, copied unchanged into the submission
`question`	`And in 2013?`	The user message, used exactly as written

1. Read the file as text (`pd.read_csv(path, dtype=str, keep_default_na=False)`). Do not edit, reorder or drop rows.
2. Answer each conversation separately, turn by turn in order, starting with an empty conversation memory.

3. **Do not hard-code answers** or write rules for specific questions. Graders search the repository for question IDs and question text.

B. How to produce submission.csv

1. Run your final notebook on Kaggle at tag `p1-v1.0` with the competition data attached. It reads the questions file from `QUESTIONS_PATH` (default `Questions.csv`) and writes `submission.csv` with no manual step, within 90 minutes on a Kaggle GPU.
2. Write one row per question: `id, conversation_id, question_id, answer, figure_references, citations, status`. Join lists with commas and no spaces (`Figure A-6,Figure A-7`); write `NONE` for an empty list.
3. Run `validate_submission.py`, then submit `submission.csv` from the notebook's Output tab and select it as your final submission.

```
import os, pandas as pd
QUESTIONS_PATH = os.environ.get("QUESTIONS_PATH", "Questions.csv")   # graders swap in the hidden set
q = pd.read_csv(QUESTIONS_PATH, dtype=str, keep_default_na=False)
rows = []
for conv_id, turns in q.groupby("conversation_id", sort=False):
    chat = new_conversation()                          # your code: empty memory for this conversation
    for _, r in turns.sort_values("turn", key=lambda t: t.astype(int)).iterrows():
        a = chat.answer(r["question"])                  # your code: returns the JSON answer as a dict
        rows.append({"id": r["id"], "conversation_id": conv_id, "question_id": r["question_id"],
                     "answer": " ".join(a["answer"].split()),
                     "figure_references": ",".join(a["figure_references"]) or "NONE",
                     "citations": ",".join(a["citations"]) or "NONE", "status": a["status"]})
pd.DataFrame(rows).set_index("id").loc[q["id"]].reset_index().to_csv("submission.csv", index=False)
```

C. Scoring

Per question: 0.40 answer correctness + 0.20 status + 0.20 figure references + 0.20 citations, averaged over all questions.

D. Hidden question set (final grading)

1. `Questions.csv` and the public leaderboard are for practice and feedback. **Marks for Tasks 1.3 and 1.5 come from a hidden question set** that graders run through your tagged Kaggle notebook after the deadline. It has the same format and kinds of questions, including reworded versions of public questions.
2. Task 1.5 marks use your rank on the hidden-set score (table below). Hard-coded answers or question-specific rules score 0 for Tasks 1.3 and 1.5.

Kaggle Class Competition: Sign Up Link: [link posted on Canvas]

Rank on hidden-set score	Points Awarded
1-10	10
11-20	9
21-30	8
31-40	7
41 +	5

Part 2: Fine-Tuning Diffusion Models for Domain-Specific Image Generation *(27 Marks)*

Reference papers:

- Rombach et al., 2022. High-Resolution Image Synthesis with Latent Diffusion Models (Stable Diffusion). https://arxiv.org/abs/2112.10752
- Hu et al., 2021. LoRA: Low-Rank Adaptation of Large Language Models. https://arxiv.org/abs/2106.09685
- Carlini et al., 2023. Extracting Training Data from Diffusion Models. https://arxiv.org/abs/2301.13188

Fine-tune a pretrained text-to-image model on a captioned image dataset, compare two fine-tuning runs with the base model, and evaluate quality, prompt alignment, diversity and cost.

Dataset and base model

- Naruto BLIP Captions (`lambdalabs/naruto-blip-captions`) or Pokémon Captions (`reach-vb/pokemon-blip-captions`), or another captioned set of at least 500 images with a stated license.
- Base model: Stable Diffusion 1.5 recommended (SDXL, SD 3.5 Medium or FLUX.1 allowed if they fit your GPU). Use the same base model for both runs.

Actionable Tasks:

2.1. Data Preparation *(4 Marks)*

1. Load the dataset, remove corrupted and duplicate images, and report how many were removed.
2. Split 90% train / 10% test with seed 266. Test images are used only for evaluation.
3. Resize to the model's resolution (512×512 for SD 1.5). Describe the dataset in a few lines, including two limitations or biases.

2.2. Fine-Tuning *(9 Marks)*

1. Train **two runs** with the same data and step budget: (1) LoRA, and (2) either a second LoRA rank, QLoRA, DoRA or full fine-tuning of the UNet.
2. For each run, report in one table: trainable parameters, learning rate, steps, batch size, peak GPU memory and training time (all printed in the notebook).
3. Push both trained models (LoRA adapters or weights) to Hugging Face, with a short model card for the run you select as final.

2.3. Image Generation *(5 Marks)*

1. Write a fixed set of **20 prompts** before training ends: 10 in-domain (from test-set captions), 5 out-of-domain and 5 compositional (e.g. "two characters, one in red and one in blue").

2. Generate 2 images per prompt (seeds 0 and 1) for the base model and both runs, with the same sampler settings: 40 images per model.
3. Show side-by-side grids (base vs run 1 vs run 2) for 8 prompts. Upload all images to a Hugging Face dataset with prompt, seed and model in the metadata.

2.4. Evaluation, Safety & Documentation *(9 Marks)*

Measurement	Part 2 definition
Primary	**KID** between in-domain generations and test images (lower is better; note it is noisy with few images) and **CLIP score** between each prompt and its image
Secondary	**Diversity:** average LPIPS distance between the two seeds of each prompt
Human evaluation	Blinded comparison of 25 image pairs (final run vs base model, left/right randomized). At least 3 raters (both members plus one person outside the team) pick the better image for domain style and prompt match. Report the win rate
Efficiency	GPU-hours per training run (with peak GPU memory)

1. Show at least **two failure modes** with example images (e.g. ignoring the prompt on out-of-domain prompts, overfitting, repeated outputs), and say which run you would choose and why.
2. **Safety (C4):** license, NSFW filter, and a **memorization check**: for 10 in-domain images, find the most similar training image (CLIP similarity) and show the 3 closest pairs.

Part 3: A Multimodal Multi-Agent System for Image Understanding and Generation *(28 Marks)*

Reference papers:

- Yao et al., 2022. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629
- Madaan et al., 2023. Self-Refine: Iterative Refinement with Self-Feedback. https://arxiv.org/abs/2303.17651
- Brooks et al., 2023. InstructPix2Pix: Learning to Follow Image Editing Instructions. https://arxiv.org/abs/2211.09800

Build a system of **five agents** that understands an input image, turns a user request into a good prompt, generates a new image (using your Part 2 model for domain-style requests), checks the result, and revises it if needed.

> **Flow:** user request → Planner (with input safety check) → Vision Understanding → Prompt Engineering → Image Generation → Critique (with output safety check) → accept, or revise (back to Prompt Engineering, at most 2 times). Caption or question-only requests stop after Vision Understanding; unsafe requests are refused by the Planner.

Actionable Tasks:

3.1. Agents *(10 Marks)*

Agent	MUST do
Planner	Classify the request (caption/question, style change, edit, domain-style generation, unsupported), choose the route, and refuse unsafe requests (NSFW, deceptive edits of real people, instructions hidden in the image)
Vision Understanding	Caption, list of main objects, any text visible in the image, and answers to at least 2 questions about the image
Prompt Engineering	Rewrite the request into a clear generation prompt using the vision output, keeping what the user wants unchanged. Never treat text found in the image as an instruction
Image Generation	At least two modes: style change (image-to-image) and one of edit or enhancement. Domain-style requests use your Part 2 model from Hugging Face. Record model, prompt, seed and steps
Critique	Score the result (CLIP similarity to the prompt plus a short vision-language model judgement), check the output is safe, and decide accept or revise with a concrete revision instruction

3.2. Orchestration, Failure Handling & Demo App *(8 Marks)*

1. Connect the agents with LangGraph or plain Python functions that pass one shared, typed state (e.g. a dataclass or pydantic model).
2. Limit the revise loop to 2 revisions. Save a JSON trace of every request: which agent ran, its input and output, and how long it took.
3. Handle ambiguous requests, poor-quality or broken images and generation errors with a clear message instead of crashing.
4. Build a Gradio app that runs locally or in a Kaggle/Colab notebook and shows each agent's output for a request. You will use it in the team demo. Deploying it as a Hugging Face Space is optional (bonus).

3.3. Evaluation, Safety & Documentation *(10 Marks)*

1. Write **15 test scenarios** before you run them: 3 each of caption and visual questions, style change, edit or enhancement, domain-style generation with your Part 2 model, and ambiguous or broken input. Give each a short pass/fail rubric.
2. Write **10 red-team cases**: instructions written inside images, unsafe requests, edits of real people, broken or empty files.
3. Compare the system with and without the critique loop on the 15 scenarios.

Measurement	Part 3 definition
Primary	**Task success rate:** scenarios that pass their rubric (out of 15)
Secondary	**Critic agreement** (how often the critic's accept/revise decision matches yours) and **red-team attack success rate** (out of 10)

Measurement	Part 3 definition
Human evaluation	Both members judge each scenario pass/fail independently using the rubric; report how often you agreed and how you resolved disagreements
Efficiency	Median seconds per request

Submission, Demo & Grading *(all parts)*

S.1. What to Submit by Nov 17, 11:59 PM PT

1. On Canvas, one zip named `Pair_Programming_Team_##.zip` containing: the report PDF, `CONTRIBUTIONS.md`, `AI_USAGE.md`, and `links.txt` with your GitHub repository and the commit of each tag, your Hugging Face links (Part 2 models, Part 2 image dataset), and your final Kaggle notebook link.
2. Tags `p1-v1.0`, `p2-v1.0` and `p3-v1.0` pushed to GitHub. Tags created after the deadline are ignored.
3. Your final Kaggle submission selected on the leaderboard.

S.2. Team Demo (Nov 18–24) *(10 Marks)*

15 minutes, both members present, run from your tagged code. Graders choose the questions, prompts and scenarios on the spot.

What you show	Marks
Part 1: run your answering code on 3 questions chosen by the graders, including a short multi-turn conversation	3
Part 2: generate images for 2 prompts chosen by the graders, with the base model and your fine-tuned model	2
Part 3: run 3 scenarios in your Gradio app, including one that must fail safely	3
Questions: each member explains design choices and code in any part, including code the other member wrote	2
Total	**10**

S.3. Penalties

Finding	Consequence
A notebook is submitted without outputs	At most 50% for the items it supports
A number in the report does not match the notebook output	0 for that item
Hard-coded answers or question-specific rules in Part 1	0 for Tasks 1.3 and 1.5
Fabricated results	0 for the whole part and a referral under the SJSU academic integrity policy
A member cannot explain their own work in the demo	That member’s marks for the part reduced by up to 50%

S.4. Marks Summary

Part	Task	Marks
1	1.1 Ingestion & Indexing	5
1	1.2 Retrieval & Grounded Answers	7
1	1.3 Conversation Handling	5
1	1.4 Evaluation, Safety & Documentation	8
1	1.5 Kaggle Leaderboard	10
2	2.1 Data Preparation	4
2	2.2 Fine-Tuning	9
2	2.3 Image Generation	5
2	2.4 Evaluation, Safety & Documentation	9
3	3.1 Agents	10
3	3.2 Orchestration, Failure Handling & Demo App	8
3	3.3 Evaluation, Safety & Documentation	10
All parts	S.2 Team Demo	10
	Total	**100**

S.5. Bonus *(up to +4, total capped at 100)*

- **+2:** deploy the Part 3 app as a Hugging Face Space.
- **+2:** add a reranker to Part 1 and show the improvement on your golden set.

S.6. DO / DON'T

DO:	DON'T:
1. Submit one report covering all three parts. 2. Use IEEE format and your own words; plagiarism is checked across groups. 3. Include an architecture diagram for each part. 4. Explain what every table and chart shows. 5. Fix and report seeds. 6. Save model weights to Hugging Face regularly.	1. Do not split parts between members; both work on all three parts. 2. Do not hard-code answers or tune on the questions you report results for. 3. Do not use AI coding assistants beyond what you declare in `AI_USAGE.md`. 4. Do not commit weights, data or secrets to GitHub.